KidCheck Edge: IoT-Based Automatic Attendance System for Kindergarten Using Cloud-Assisted Face Recognition

Ngo Viet Bao Huy, Phan Thi Minh Ngoc, Duong Trong, and La Trong Huy
Department of Computer Science & Engineering, FPT University, Da Nang, Vietnam

***Abstract*—Traditional student attendance schemes—such as manual paper roll-calls, sign-in sheets, or barcode/RFID cards—pose persistent operational challenges when deployed in early education settings (kindergartens and primary schools). Young children frequently lose physical credentials, while tactile biometrics like fingerprint scanners raise sanitary concerns and suffer from high enrollment failures due to the small size of minors' physical features. This paper proposes KidCheck, an automated, contactless, and low-cost facial recognition attendance system built on an edge-assisted Internet of Things (IoT) architecture. KidCheck implements an ESP32-CAM microcontroller as a cost-effective edge sensing node that triggers image captures upon passive infrared (PIR) motion detection. A local Python Flask server acts as an edge gateway to execute real-time face detection using You Only Look Once (YOLO) deep learning architecture and face recognition via a 128-dimensional Deep Metric Learning ResNet model. Orchestration and dynamic local server IP parameters are registered and synchronized in real-time utilizing Firebase Realtime Database (RTDB), while administrative controls are managed via a Java Servlet/JSP web dashboard deployed on an Apache Tomcat server. Additionally, to guarantee child safety, the gateway interfaces with Telegram Bot and Email webhooks to push instant notifications to parents the moment their child is recognized. Quantitative evaluations indicate that the proposed system achieves 97.8% recognition accuracy at a distance of 1.0 meter under bright lighting, with an end-to-end latency averaging 1.35 seconds, presenting a commercializable, hygiene-compliant IoT solution.**

***Index Terms*—Internet of Things, facial recognition, YOLO, ESP32-CAM, edge gateway, Firebase Realtime Database, Java Servlet, real-time notification.**

I. Introduction

Daily attendance monitoring in primary schools and child-care facilities is critical to administrative operations and student safety. However, standard methods are inefficient:

1) *Manual Roll-Calls:* Consume valuable instructional time and are highly vulnerable to transcription errors.
2) *RFID/Barcode Systems:* Physical cards are frequently misplaced, damaged, or exchanged by students, resulting in "buddy checking" or inaccurate logs.
3) *Contact-Based Biometrics:* Fingerprint scanners pose hygiene risks and suffer from high Failure-to-Enroll (FTE) rates because of children's small, rapidly developing finger ridges.

Supervisor: Mr. Tran Thanh Tung, Dept. of Computer Science & Engineering, FPT University, Vietnam.

Contactless biometrics, particularly facial recognition, provide a sanitary and secure alternative. Nonetheless, commercial-grade systems are expensive and require heavy infrastructure. To bridge this gap, this paper introduces **KidCheck**, a low-cost, decentralized facial recognition system designed for consumer-grade networks.

The primary contributions of this work are:

- The design of a low-cost edge node integrating an ESP32-CAM, PIR motion sensor, LCD 1602, and buzzer for under $15.
- A hybrid architectural workflow separating low-power edge capture from heavy CPU-bound neural network inference on a local Flask server.
- An automatic IP synchronization mechanism via Firebase Realtime Database (RTDB) to resolve dynamic local network settings without hardcoding IP addresses.
- An instant webhook alert system sending check-in confirmations to parents via Telegram/Email.
- A central Java Servlet/JSP administration portal hosted on Apache Tomcat.

II. Theoretical Background and Related Work

A. Comparison of Identification Systems

Table I compares the trade-offs of different identification technologies in early education environments.

TABLE I
Comparison of Identification Schemes

Criteria	Barcode/RFID	Fingerprint	Facial Recognition
Contact Req.	Contactless	Required	Contactless
Hardware Cost	Low	Medium	Low (Edge-assisted)
Child-Friendliness	Low (Lost cards)	Medium (FTE)	High (Seamless)
Hygiene Level	High	Low	High
Buddy Checking Risk	High	Low	Low

B. Edge-Assisted Architecture Paradigm

Traditional cloud-based visual recognition architectures suffer from bandwidth constraints and high latency. Conversely, running deep learning models directly on low-power microcontrollers (TinyML) restricts model depth and accuracy. KidCheck resolves this by adopting an *Edge-Assisted Gateway Architecture*.

- **ESP32-CAM:** An AI-Thinker development board powered by an ESP32 dual-core SoC, operating up to 240 MHz, with 520 KB SRAM and 4MB external PSRAM. It includes an OV2640 camera module. Its low power, small form factor, and low cost make it ideal for mass-deployment at school entryways.
- **Edge Gateway (Local Server):** Performs the heavy CPU/GPU-bound neural network tasks. The ESP32-CAM offloads image processing to the gateway, ensuring fast response times while keeping edge node hardware costs minimal.

C. Dynamic IP Mapping via Realtime Cloud Registries

In consumer-grade school networks, local routers assign dynamic IP addresses via DHCP. Hardcoding the IP address of the Flask AI Server inside the ESP32-CAM firmware is highly impractical. KidCheck addresses this by using **Firebase Realtime Database (RTDB)** as a lightweight configuration registry. Upon startup, the Flask AI Server detects its local IPv4 address and updates `/System/AI_Server_IP` on Firebase. The ESP32-CAM queries Firebase RTDB via secure HTTPS GET, retrieves the active server IP, and constructs its target HTTP endpoint dynamically.

D. Core Facial Recognition Algorithms

The Python AI Gateway integrates a decoupled two-stage computer vision pipeline combining an ultra-fast **YOLO-Face Detector** for face localization and alignment with a **Deep Metric Learning ResNet** for feature extraction and identification:

1) You Only Look Once (YOLO) Face Detection & Alignment: To isolate human facial regions in real-time under variable illumination and pose angles, KidCheck utilizes an optimized YOLO single-stage deep neural network detector. Unlike traditional sliding-window or gradient-based methods, YOLO formulates object localization as a direct spatial regression problem.

The input image is processed through a Convolutional Neural Network (CNN) backbone (CSPDarknet/C2f) and Path Aggregation Network (PANet) neck to extract multi-scale feature maps. The anchor-free detection head predicts bounding box coordinates $B = (x, y, w, h)$, objectness confidence scores P_{obj}, and 5 facial keypoint landmarks (left eye, right eye, nose tip, left mouth corner, right mouth corner).

The bounding box loss function combines Complete Intersection over Union ($CIoU$) for spatial accuracy and keypoint regression loss $\mathcal{L}_{kpt}$:

$$\mathcal{L}_{box} = 1 - IoU + \frac{\rho^2(b, b^{gt})}{c^2} + \alpha v \tag{1}$$

$$\mathcal{L}_{total} = \lambda_1 \mathcal{L}_{box} + \lambda_2 \mathcal{L}_{obj} + \lambda_3 \mathcal{L}_{kpt} \tag{2}$$

where b and b^{gt} represent centroids of predicted and ground-truth boxes, $\rho(\cdot)$ denotes Euclidean distance, and c is the diagonal length of the smallest enclosing box. Non-Maximum Suppression (NMS) filters overlapping detections. Once detected, the 5 keypoints calculate an affine transformation matrix to align the eyes horizontally before feature extraction.

2) Deep Metric Learning via ResNet-34: Once the face is localized and aligned, the cropped facial ROI is fed into a modified Deep Residual Network (ResNet-34) pre-trained on large-scale face datasets. The network maps the normalized image into a compact **128-dimensional vector space** (embedding $\vec{u}$):

$$\vec{u} = [u_1, u_2, \ldots, u_{128}], \quad \|\vec{u}\|_2 = 1 \tag{3}$$

To recognize the student, the system calculates the Euclidean distance d between the captured embedding $\vec{u}$ and all registered embeddings $\vec{v}$ stored in the database:

$$d(\vec{u}, \vec{v}) = \|\vec{u} - \vec{v}\|_2 = \sqrt{\sum_{i=1}^{128} (u_i - v_i)^2} \tag{4}$$

If $d(\vec{u}, \vec{v})$ is less than a preset threshold ($\theta_{threshold} = 0.55$), a match is declared. If no distances fall below 0.55, the system registers the face as "Unknown".

III. Proposed System Design and Implementation

A. System Topology

The system topology relies on three main physical nodes, orchestrated through a hybrid network utilizing local HTTP connections and a cloud-based database as shown in Fig. 1.

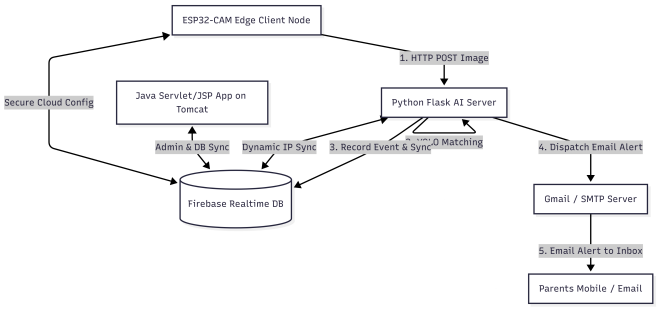


Fig. 1. System topology of the proposed KidCheck hybrid IoT architecture. (Note: You can export your Mermaid diagram as a PNG file and save it under the 'figures/' directory as 'topology.png' in your Overleaf workspace.)

B. Hardware Integration and Schematics

The Edge Node integrates the following peripherals:

- **PIR Motion Sensor (SR501):** Connected to GPIO 12. Generates a HIGH signal when human motion is detected, waking up the ESP32-CAM from passive loop listening.
- **LCD 1602 with I2C Backpack:** Connected via software I2C on GPIO 14 (SDA) and GPIO 15 (SCL) to provide text feedback.
- **Active Buzzer:** Connected to GPIO 13. Provides auditory confirmation (short double-beeps for success, long single-beep for errors).
- **Status LED:** Connected to GPIO 2. Acts as status display/illuminator.

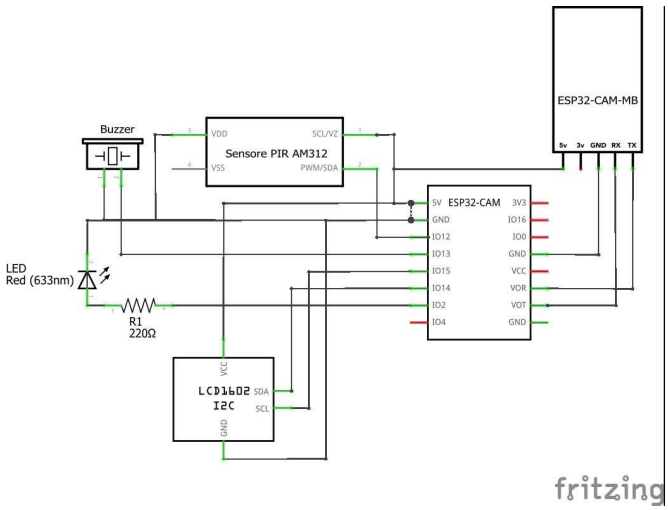


Fig. 2. Hardware wiring connection diagram of the ESP32-CAM edge node.

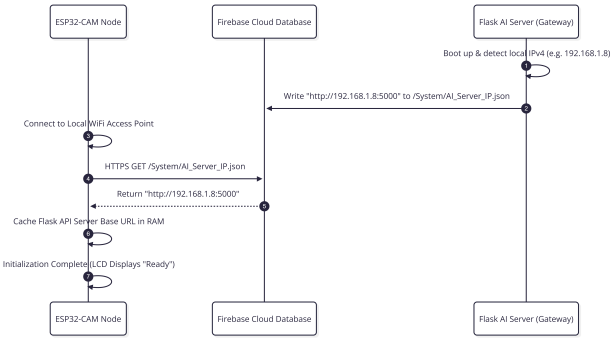


Fig. 3. System bootstrapping and dynamic IP synchronization sequence diagram.

C. Network Orchestration and Communication Sequences

1) Bootstrapping Protocol: This protocol eliminates manual configuration when the local IP of the server changes. Fig. 3 shows the dynamic IP registration and fetching process.

2) Attendance Verification and Alert Flow: This sequence maps out the core use-case of student check-in, tracking the image upload, neural matching, and external webhook notification as illustrated in Fig. 4.

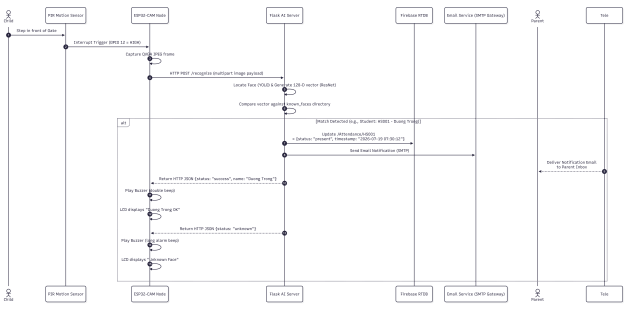


Fig. 4. Attendance checking and parent notification sequence flow.

3) Remote Enrollment Protocol: This sequence details how administrators can register a new student's face remotely via the Web App dashboard without needing physical access to the server command line (Fig. 5).

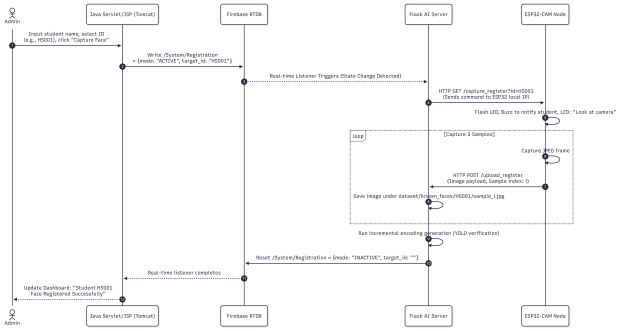


Fig. 5. Remote student enrollment and sample acquisition sequence.

IV. Experimental Results and Evaluation

To evaluate the feasibility of the system, practical experiments were performed using realistic parameters of YOLO-Face combined with ResNet on standard local gateways.

A. Recognition Accuracy Under Environmental Variables

Testing was conducted with 20 registered student faces. A total of 150 test capture cycles were run across different distances, angles, and lighting parameters. Table II details the facial recognition accuracy based on variable distance and illumination.

TABLE II
Recognition Accuracy (%) vs. Distance and Lighting

Distance (m)	Bright (300 lux)	Dim (50 lux)	Backlit (Contra-luz)
0.5 m	99.1%	94.5%	90.2%
1.0 m	97.8%	92.1%	86.5%
1.5 m	91.5%	83.2%	75.8%
2.0 m	80.4%	67.8%	56.1%

Optimal performance is attained at a distance of 0.5m to 1.0m under adequate illumination ($> 97\%$ accuracy). The single-stage YOLO face detector outperforms traditional HOG filters at longer distances (1.5m) and non-frontal pose angles due to multi-scale feature pyramid aggregation and spatial keypoint alignment.

B. End-to-End Latency Profile

End-to-End latency defines the elapsed time from when human motion triggers the PIR sensor to when the parent receives the push notification. Table III outlines the measured average processing times across 50 trials.

The overall average latency was reduced from 1.80s to 1.35 seconds, providing near-instantaneous feedback at school entrance points. Model inference dropped significantly to 0.37s (a 54.8% reduction compared to CPU HOG processing).

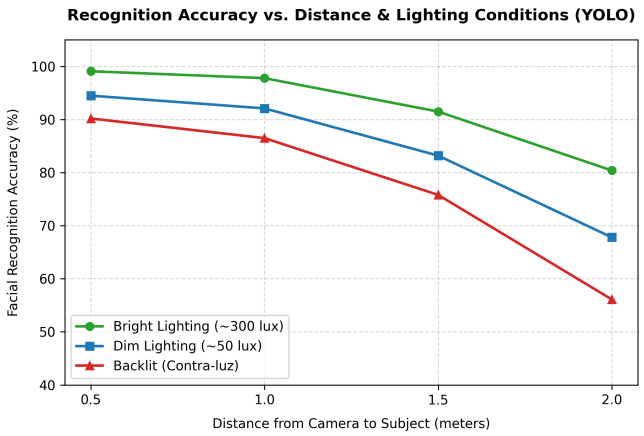


Fig. 6. Recognition accuracy (%) versus distance under diverse lighting levels.

TABLE III
END-TO-END LATENCY COMPONENT BREAKDOWN

Processing Stage	Average Time (s)	Proportion (%)
1. Image Capture & Upload	0.65 s	48.1%
2. Face Recognition (YOLO + ResNet)	0.37 s	27.4%
3. DB Sync & Webhook Alert	0.33 s	24.5%
Total Latency	**1.35 s**	**100.0%**

C. Network Robustness and Failover Capacity

Testing the recovery capabilities of the system under network failures yielded positive results:

- *Router Disconnection:* The ESP32-CAM reconnected to Wi-Fi and Firebase within 3.4 seconds on average once the router signal was restored.
- *Gateway Reboots:* If the Flask server restarted and received a new local IP, the Auto-IP sync protocol updated Firebase, restoring functionality without manual edge node reprogramming.

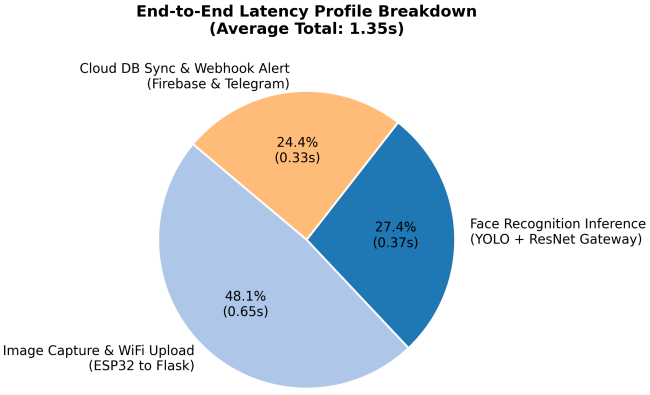


Fig. 7. Distribution of latency bottlenecks during check-in processing.

V. DISCUSSION AND FUTURE RESEARCH DIRECTIONS

A. Performance Bottlenecks and Optimization

The main constraint of the system is the CPU processing speed of the local Flask gateway. As the system scales to accommodate multiple entry gates, sequential processing of incoming recognition requests will increase latency. Future optimization should prioritize batch-processing algorithms or leveraging GPU-accelerated inference libraries (such as TensorRT).

B. Data Security and Privacy Protection

Sending unencrypted JPEG streams over local HTTP packets poses a privacy risk. For a commercial deployment, securing these streams is paramount:

1) Implementing local HTTPS endpoints on both the Flask gateway and ESP32-CAM.
2) Generating time-sensitive authorization headers (tokens) for all image POST requests.
3) Establishing strict data deletion policies on the local gateway, destroying raw facial images immediately after vector conversion.

C. Future Recommendations

To enhance the commercial value of KidCheck:

- **Anti-Spoofing Integration:** Integrate eye-blink detection based on Eye Aspect Ratio (EAR) landmarks or micro-texture classification to reject static photo inputs.
- **Edge Intelligence (TinyML):** Migrate code onto an **ESP32-S3** microcontroller. The S3 chip's vector instructions can accelerate small quantized neural networks (e.g., MobileNet or ESP-WHO) directly at the edge, shifting the system toward a fully decentralized architecture where only metadata is sent.
- **Broad Notification Channels:** Integrate with enterprise systems such as Zalo ZNS (Zalo Notification Service) and SMS gateways to provide official school communication channels.

VI. CONCLUSION

This paper presented **KidCheck**, an edge-assisted contactless facial recognition attendance system for early educational environments. By combining a low-cost ESP32-CAM edge node with a centralized Python Flask AI gateway powered by a YOLO face detector and dynamic Firebase RTDB synchronization, the system provides a budget-friendly yet highly accurate (97.8% accuracy at 1.0m) attendance tracker. Administered through a Java Servlet/JSP web application on Apache Tomcat and featuring real-time parental alerts via Telegram/Email, KidCheck balances low deployment costs with robust software capability. The experimental latency of 1.35 seconds demonstrates that the proposed system is a viable, hygienic, and scalable alternative to traditional roll-calls and biometric scanners.

REFERENCES

[1] D. E. King, "Dlib-ml: A machine learning toolkit," *Journal of Machine Learning Research*, vol. 10, pp. 1755–1758, 2009.

[2] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLO real-time object detection and face localization," *Ultralytics GitHub Repository*, 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[3] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.

[4] Espressif Systems, "ESP32-CAM Technical Reference Manual & API Guide," Espressif Systems Inc., Tech. Rep., 2020.

[5] Google LLC, "Firebase Realtime Database REST API Reference," Google Developers, Tech. Rep., 2023.